

A Persistent MCP Harness for OpenClaw to Play Pokémon Showdown

Architecture, Decision Models, and Conditional Mathematical Guarantees

`pokemon-showdown-player`

Technical report grounded in the repository implementation

7 October 2026

Source snapshot: `151393d`; feature schema 1

Abstract

This report explains a working connection between OpenClaw, a Model Context Protocol (MCP) server, and Pokémon Showdown. OpenClaw coordinates a persistent player process; a local action-conditioned neural actor-critic selects complete singles or doubles decisions. The harness constructs candidate actions from the player’s request, records sampled decisions and verified terminal outcomes, and hands learning to a bounded background worker. The worker trains with proximal policy optimization (PPO), evaluates a separate candidate, and may atomically replace the deployed checkpoint. Two additional models predict publicly executed opposing moves and select validated team versions from outcome evidence.

We give the exact neural architecture, losses, feature construction, data contracts, and evaluation rule implemented in the source. We prove conditional properties of information flow, joint action constraints, masked policies, policy gradients, importance ratios, generalized advantage estimation, and Bayesian team updates. We also derive a statistical improvement certificate that the current empirical promotion rule does not implement. Worked examples connect the equations to battle decisions. The report describes a system and its limitations; it does not claim optimal play, calibrated win probabilities, or monotonic improvement.

Keywords: MCP, OpenClaw, Pokémon Showdown, doubles decisions, partial observation, action masking, PPO, Thompson sampling.

Contents

1	Scope and how to read this report	3
2	Architecture: from a tool call to a battle decision	3
3	Mathematical description of the battle	5
4	Candidate actions and doubles constraints	6
5	Feature construction and the tactical prior	8
6	The action-conditioned actor–critic	9
7	Learning from outcomes: policy gradients, GAE, and PPO	11
8	The public-move scout	15
9	Team choice: an outcome bandit over exact versions	16
10	Background learning and the deployed model	18
11	Evaluation: what would establish improvement?	19
12	Limitations and concrete research directions	21
A	Notation glossary	21
B	Source-to-equation audit map	22
C	Reproducibility and document build	23

1 Scope and how to read this report

The practical question is: how can an agent operate a real Showdown player while delegating battle decisions to a persistent, trainable local model? The repository answers it with a separation of responsibilities. The agent coordinates tools; the harness handles the connection and battle lifecycle; the model chooses actions; the learner proposes and evaluates updates. These components have different kinds of evidence and different failure modes.

This paper describes the source at commit `151393d` in the local repository, not an arbitrary future deployment. Its main target is `gen9championsvgc2026regmc`. The simulator dependency is pinned to official Showdown commit `c046106cbe075931b1ff8d8b800ff5be47a85f96`. The exact format, simulator version, feature schema, team fingerprint, and collecting model revision are part of the scientific description. In particular, an identifier beginning with `gen9` is not sufficient to determine all battle mechanics. The pinned Champions rules use their own stat allocations and Mega Evolution mechanics; the live server remains the authority for live legality [10, 16].

Three kinds of statements are kept distinct throughout:

1. **Implemented behavior:** statements directly traceable to repository functions, such as the dimensions of the neural network.
2. **Conditional mathematical results:** statements proved under explicit assumptions, such as normalization of a nonempty masked softmax.
3. **Evaluation proposals:** stronger checks that would support future claims but are not present in the current runtime.

Sections 2–4 explain the system and battle representation. Sections 5–7 derive the decision model and learning equations. Sections 8–11 cover scouting, teams, and the meaning of evaluation. The appendices provide a symbol glossary, a source map, and reproducibility instructions. Each proof is followed by its practical meaning or scope.

2 Architecture: from a tool call to a battle decision

2.1 What MCP contributes

MCP supplies a standard interface for discovering tools and calling them with structured arguments. Its stdio transport carries JSON-RPC messages between a client and a subprocess [5, 6]. In this project, `ps_mcp_server.py` creates the server, registers functions as tools, and returns their outputs as strings, frequently containing JSON. MCP itself does not supply Pokémon mechanics, a battle policy, a learning algorithm, or a win-rate guarantee. Those are application responsibilities.

OpenClaw is the MCP client and orchestration layer. Its native registration can expose this server’s tools as `pokemonshowdown__ps_*`; the server registry name and the internal MCP server name need not be identical. OpenClaw’s documented process lifecycle matters: a retained session can keep downstream MCP connections between turns, whereas a detached one-shot run retires its connections at run end [7]. A later turn in a new process cannot assume the old socket or in-memory episode exists.

2.2 Connection and execution boundary

One module-level `PSCClient` is shared with `Player`. The client opens a `WebSocket`, receives a challenge string, exchanges configured credentials for a login assertion, and sends the assertion back through Showdown’s login command. Credentials stay outside model observations and the paper’s

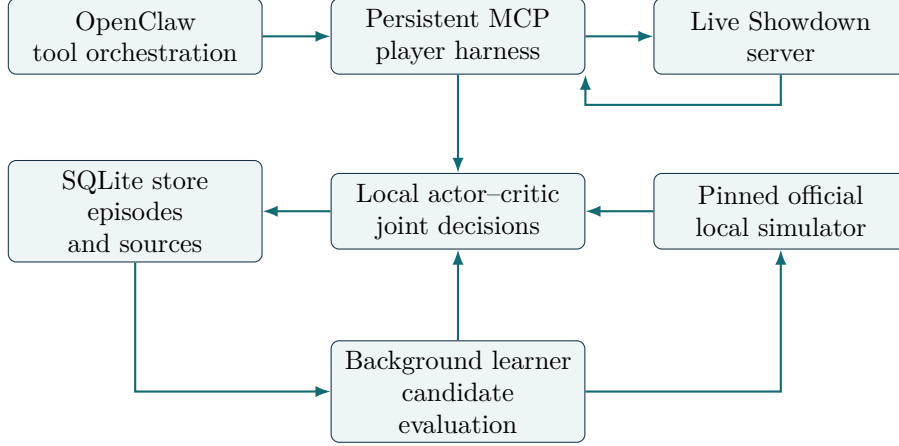


Figure 1: Application responsibilities. Tool calls and live play share one player connection. Local simulation needs no public account. The worker prepares a candidate separately; the upward path changes the deployed actor only after the promotion conditions hold. Research and scouting inputs also enter the brain but are omitted from this overview for clarity.

artifacts. Room messages have the form `ROOMID|TEXT`; a battle decision becomes `ROOMID|/choose CHOICE`. The application buffers room messages and maintains full battle logs separately from the bounded recent-room buffers [9].

The socket reader receives public events and the player’s own private request. The request describes available moves, move slots, active positions, switching information, and the player’s party. The harness combines that request with the visible protocol prefix to construct a perspective-specific observation. It does not give the actor an omniscient simulator state.

The strongest normal workflow is:

```

ps_team_plan(format, save_as="Planned team")
ps_login()
ps_ml_ladder(format, team="Planned team")
ps_ml_wait(timeout=120) # use returned room and exact selected team
ps_ml_play(room, format, team, learn=True)
ps_ml_finish(room)
ps_learning_status()

```

These are application tool names, not language syntax required by MCP. Explicit team selection may still choose a researched variant: the returned team identity must be used to attribute the game correctly. `ps_ml_play` handles the request loop inside the same server process. An outer agent timeout must be long enough for the battle-owning call; the MCP timeout alone cannot extend the host agent’s lifetime.

2.3 Recommendation, submission, and learning are distinct operations

The asynchronous model lock covers individual choices and finalization. A per-room lock prevents two simultaneous full-game loops for the same room. Long battles release the model lock between decisions so status and research calls remain available. A background observer can finalize terminal games. One implementation detail is relevant to experiments: `train=False` on `ps_ml_finish` does not suppress the ordinary background enqueue inside `LiveSession.finish`. Disabling learning requires control of the learning configuration and collection behavior, not only that final tool

Operation	Meaning
<code>ps_ml_decide</code>	Compute a recommendation and policy probabilities without submitting or recording a PPO sample.
<code>ps_ml_choose</code>	Sample and submit a model decision with <code>explore=True</code> ; record its collecting distribution. Deterministic mode does not collect PPO data.
<code>ps_choose</code>	Submit a supplied action through the basic connection. No collecting model distribution is established automatically.
<code>ps_ml_demonstrate</code>	Submit a supplied candidate as an explicit behavior-cloning label. Use one collection mode for the whole recorded game.
<code>ps_ml_finish</code>	Verify a terminal outcome, finalize an episode, and expose background learning status.
<code>ps_ml_train</code>	Queue candidate learning; do not immediately replace the deployed actor.

Table 1: The collection contract prevents a recommendation or arbitrary override from being silently treated as a sampled policy action.

argument.

Implementation: `ps_mcp_server.py`; `PSCClient` in `ps_client.py`; `Player` in `harness.py`; `LiveSession` in `ml/live.py`.

3 Mathematical description of the battle

3.1 A partially observed stochastic game

Let s_t denote the complete battle state at a decision boundary. It includes both teams’ hidden information, current HP, statuses, field conditions, pending mechanics, and any other variables required by the simulator. Player 1 chooses a complete action a_t , player 2 chooses b_t , and randomness ξ_t affects the transition:

$$s_{t+1} \sim P(\cdot \mid s_t, a_t, b_t), \quad o_t^i \sim O_i(\cdot \mid s_t). \quad (1)$$

For exposition this is a simultaneous decision phase. A real trace also contains preview, forced replacement, and waiting phases. The actual implementation index t counts recorded actionable requests by the learner; it is not necessarily the battle turn number.

This is naturally a two-player partially observed stochastic game. Holding the opponent’s strategy fixed gives a single-agent controlled process. If that opponent is history-dependent, a sufficient latent state must include its relevant memory; simply averaging P over an opponent distribution dependent only on s_t would not capture every opponent.

Write h_t for the player’s available interaction history before decision t , including its own requests and past actions. An exact Bayesian belief would be

$$\beta_t(s) = \mathbb{P}(s_t = s \mid h_t). \quad (2)$$

For a fixed opponent and a sufficient latent state, with joint transition kernel $\bar{P}$ and observation kernel O , its schematic update is

$$\beta_{t+1}(s') = \frac{O(o_{t+1} \mid s') \sum_s \bar{P}(s' \mid s, a_t) \beta_t(s)}{\sum_u O(o_{t+1} \mid u) \sum_s \bar{P}(u \mid s, a_t) \beta_t(s)}. \quad (3)$$

The denominator must be positive. Equation (3) explains what a full belief filter would require. The repository does not implement this filter or enumerate hidden teams. It instead computes a

compact observation representation $x_t = \phi(h_t)$ and supplements it with public-move predictions. Partial-observation foundations are discussed in Kaelbling et al. [4].

Interpretation. A visible opponent with 50% HP is an observation. Its unrevealed item is a hidden variable. A prediction that it will use Protect is a learned forecast, not a recovered hidden state.

3.2 Information available to the model

The reconstructed state contains both own actives, the own party and bench, both visible opposing actives, revealed opposing members, preview species, actually disclosed team sheets, weather, field and side conditions, statuses, boosts, and the most recent sixteen public move events. Historical discoveries are available only after they occur in the input prefix. Open team sheets are used only when disclosed; local simulation explicitly reveals them under the format’s open-sheet rule.

Proposition 3.1 (A conditional non-anticipation property). *Fix the encoder, cached dex, and auxiliary model/data snapshots. If the request and protocol prefix supplied at decision t contain only information available to the player by t , then the constructed state and action features are functions of available information and those snapshots. Two inputs with the same request, public prefix, and auxiliary snapshots produce the same features, irrespective of future events or unrevealed opposing information.*

Proof. **observe** scans only the supplied prefix and merges the supplied own request. It does not query an opposing private request or future log suffix. Feature construction is deterministic in that observation, the candidate string, the dex, and supplied knowledge. Scout inference is a function of a filtered public prefix and its current parameters. Composing these functions proves the claim. $\square$

The input condition is essential. Feeding an omniscient log to a function called **observe** would invalidate the premise. The local bridge uses separate `getPlayerStreams().p1/p2` streams for observations. Its omniscient stream initializes the simulator but is not passed to the policy [8, 11]. Chronological leakage through a model trained on later data is a separate concern: the proposition does not certify a historical experiment’s train/test split.

The observation is also approximate. The parser covers a selected subset of protocol events rather than every special mechanic. Missing condition data maps to full HP in `hp_fraction`, and dex base stats may substitute for unknown actual stats. Missing fields generally contribute no hashed token; there is no complete explicit unknown-value mask. Moreover, historical observed opponent moves are stored as `moves_known`, while `Features._mon` encodes `moves`; recent moves still enter the history features, but those two fields are not interchangeable. Thus perspective correctness does not imply an exact or sufficient representation of all mechanics.

4 Candidate actions and doubles constraints

4.1 Actions are complete player decisions

For an actionable request r , define $\mathcal{A}(r)$ as the list generated by `legal_choices(r)`. In singles it may contain move or switch choices. In doubles, one action is a complete pair

$$a = (u_1, u_2), \tag{4}$$

where u_i specifies a move slot and target, a switching destination, or a phase-appropriate pass. The joint action here refers to one player’s two active Pokémon; the opponent’s simultaneous decision is not part of this candidate. For example:

```

move 1 1, move 2 -1
move 1 2 mega, switch 4

```

Positive targets refer to opposing active positions; negative targets refer to own positions. A move that attacks an ally can be syntactically valid even when tactically undesirable. Some scripted or locked moves omit an explicit target. The request, not a fixed move vocabulary, determines each move slot’s meaning.

The enumerator forms per-position options, takes their Cartesian product, and removes shared-constraint violations. It filters disabled or exhausted moves, handles visible trapping, and accounts for fainted or commanding positions. At most two active positions are supported. Hidden restrictions can still cause a server rejection; $\mathcal{A}(r)$ is *request-visible admissibility*, not a proof that every candidate will be accepted by the server.

Proposition 4.1 (Constraints enforced by joint enumeration). *Every emitted doubles combination has distinct switching destinations, at most one Mega-family activation, and at most one Terastallization activation. In a forced-replacement request, the number of switching positions is exactly*

$$\min(\text{forced positions, available bench members}).$$

Proof. For each Cartesian-product element the code first compares the list of switch strings with its set, rejecting duplicates. It then counts switching components against the required replacement count when forced switching is present. Finally it rejects combinations containing more than one Mega-family event or more than one Terastallization event. An emitted element has passed all these predicates. $\square$

This result concerns those predicates only. Generic support for Terastallization does not enable it in a format whose requests do not offer it. Team validation and additional hidden mechanics remain outside the proposition.

4.2 Preview is a structured action

If n party members are available and k must be brought, preview actions are ordered selections without replacement:

$$|\mathcal{A}_{\text{preview}}| = \frac{n!}{(n-k)!}. \quad (5)$$

For six members and four selected positions, the implementation enumerates $6 \cdot 5 \cdot 4 \cdot 3 = 360$ candidates. Ordering matters to the representation, including which members lead. The model scores the entire candidate rather than selecting each slot independently. There is no pruning of the preview list.

4.3 Why independent decisions can lose coordination

An independently sampled pair has probabilities $\pi(u_1, u_2 \mid x) = p(u_1 \mid x)q(u_2 \mid x)$ before any constraint handling. This factorization restricts its expressiveness.

Proposition 4.2 (An interaction a product policy cannot represent). *For two actions A, B per position, an unmasked independent product distribution cannot give positive mass to both (A, A) and (B, B) while assigning zero mass to both (A, B) and (B, A) .*

Proof. Positive diagonal probabilities imply $p(A), q(A), p(B), q(B) > 0$. Their cross products are therefore also positive, contradicting the required zeros. Equivalently, every product table has determinant zero, whereas a positive diagonal table has a positive determinant. $\square$

Interpretation. A player may want both actives to attack together or both to protect, while avoiding the mixed plans. Joint scoring can distinguish these combinations. The proposition does not say this particular neural encoder can represent every joint distribution: feature collisions and limited network capacity still constrain it.

Masking a product distribution with a shared joint constraint can itself introduce dependence, and autoregressive policies can represent dependencies too. The implemented choice is direct scoring of feasible complete candidates; the comparison above is with an unmasked independent product, not all alternatives.

Implementation: `legal_choices`, `_targets`, and `observe` in `battle_state.py`; `Features.action` in `ml/features.py`.

5 Feature construction and the tactical prior

5.1 A fixed-dimensional representation of variable objects

The state vector has $d_s = 384$ coordinates. Each action vector has $d_a = 192$. For a named feature f with numerical value v_f , signed hashing adds

$$\phi_j += \sigma(f)v_f, \quad j = H(f) \bmod (d - 16), \quad \sigma(f) \in \{-1, +1\}. \quad (6)$$

The code derives H and σ from an eight-byte BLAKE2b digest. It reserves the last sixteen coordinates for numerical features. Strings are namespaced by role and position, such as own active versus opposing active. HP fractions, normalized boosts and stats, move PP, statuses, species, items, abilities, and field conditions enter this representation. Recent public move events are weighted by $1/\sqrt{i+1}$ when counted backward from the newest event.

Most coordinates are clipped to $[-5, 5]$. Reserved state entries include turn progress, total own party HP divided by six, and candidate count divided by 1000. Hashing makes construction independent of a growing categorical vocabulary but allows collisions. The scout adds its forecast features after the state encoder’s clipping step, so the *final* augmented state should not be described as invariably confined to that original interval.

Proposition 5.1 (Hashing does not preserve all distinctions). *A signed hash map from m independent categorical coordinates into $d < m$ coordinates is not injective. Subsequent coordinate clipping cannot restore the lost distinctions.*

Proof. The linear map has rank at most d , so its null space is nontrivial. Distinct vectors differing by an element of that null space have identical hashes. Any later function, including clipping, maps identical inputs to identical outputs. $\square$

This is a representation limitation rather than a claim that a specific pair of real battle states collides. Random-hashing expectation results require random hash assumptions [18]; this implementation uses a fixed digest and does not establish such a theorem for its realized feature vocabulary.

5.2 The prior is an interpretable score, not damage

For a damaging move m against a visible target j , the prior uses the rough pressure term

$$p(m, j) = \frac{\text{basePower}(m)}{100} \text{accuracyFraction}(m) \text{STAB}(m) \text{typeMultiplier}(m, j). \quad (7)$$

STAB is 1.5 when the move’s type matches a cached species type, and otherwise 1. The multiplier is the product of target-type effectiveness values. The pressure is summed over selected live targets; allied-target pressure changes sign. Each switch contributes -0.4 . Protect, Detect, and Wide Guard each add 0.15. The final sum is clipped to $[-4, 4]$ and stored in the final action coordinate, which we denote $q_0(x, a)$.

Example 5.2 (A prior calculation). *With base power 80, accuracy 1, STAB 1.5, and effectiveness 2 against one target, the pressure is $0.8 \cdot 1 \cdot 1.5 \cdot 2 = 2.4$. A paired Protect adds 0.15, giving a joint prior of 2.55 before clipping. Pairing that attack with a switch instead gives 2.0. These numbers illustrate initialization; they are neither HP damage nor probabilities of winning.*

The prior omits actual offensive and defensive stat ratios, many abilities and items, detailed speed order, spread modifiers, protect interactions, and many format-specific rules. A high score can prefer an ineffective strategic plan. Status support generally receives zero pressure unless specially handled. At preview there is no such tactical score, so its initial prior is zero.

Implementation: `STATE_DIM`, `ACTION_DIM`, `Features.add`, `Features.state`, and `Features.action` in `ml/features.py`.

6 The action-conditioned actor–critic

6.1 Exact architecture

Let $x \in \mathbb{R}^{384}$ be the state vector and $z_a \in \mathbb{R}^{192}$ the feature vector for candidate a . The network computes

$$e_s = \tanh(W_s x + b_s) \in \mathbb{R}^{128}, \quad (8)$$

$$e_a = \tanh(W_a z_a + b_a) \in \mathbb{R}^{96}, \quad (9)$$

$$\ell_\theta(x, a) = w_\pi^\top \tanh(W_\pi [e_s; e_a] + b_\pi) + c_\pi + q_0(x, a), \quad (10)$$

$$V_\theta(x) = w_V^\top \tanh(W_V e_s + b_V) + c_V. \quad (11)$$

The actor’s hidden layer has width 96; the critic’s has width 64. The shared state encoder receives gradients from both heads during PPO. The parameter count is

$$(384 \cdot 128 + 128) + (192 \cdot 96 + 96) \quad (12)$$

$$+ (224 \cdot 96 + 96) + (96 + 1) + (128 \cdot 64 + 64) + (64 + 1) = 97,826. \quad (13)$$

The scorer is reused for every candidate; changing the candidate count does not change the learned parameter count. It uses a feedforward representation of the summarized history, not a transformer, recurrent network, or tree search.

The final actor weights and bias are initialized to zero. Consequently the initial logits are exactly $q_0(x, a)$. The other encoders and critic are initialized normally. A fresh actor is therefore a softmax over the tactical prior, with a uniform preview distribution when all preview priors are zero. Deterministic argmax tie-breaking chooses the first maximizer in the candidate order.

6.2 Masked policy

The ideal policy over a nonempty candidate set is

$$\pi_{\theta}(a \mid x, \mathcal{A}) = \begin{cases} \frac{\exp \ell_{\theta}(x, a)}{\sum_{u \in \mathcal{A}} \exp \ell_{\theta}(x, u)}, & a \in \mathcal{A}, \\ 0, & a \notin \mathcal{A}. \end{cases} \quad (14)$$

Inference scores only actual candidates. Minibatches have different candidate counts, so training pads action arrays and masks padded logits. The code uses the minimum finite floating-point value rather than literal $-\infty$. For normal finite logits this underflows to zero padded probability; tests cover that behavior. The exact support theorem below uses the ideal mathematical mask, not a universal numerical claim for pathological inputs.

Proposition 6.1 (Normalization, support, and score derivative). *For finite logits and a nonempty, parameter-independent set $\mathcal{A}$, the ideal masked policy sums to one, gives zero mass outside $\mathcal{A}$, and satisfies*

$$\nabla_{\theta} \log \pi_{\theta}(a \mid x, \mathcal{A}) = \nabla_{\theta} \ell_{\theta}(x, a) - \sum_{u \in \mathcal{A}} \pi_{\theta}(u \mid x, \mathcal{A}) \nabla_{\theta} \ell_{\theta}(x, u), \quad a \in \mathcal{A}. \quad (15)$$

Proof. The denominator is a positive finite sum, and dividing that same sum by itself normalizes the admissible probabilities. Outside probabilities are defined to be zero. Taking the logarithm gives a logit minus a log-sum-exp. Differentiating the latter gives the probability-weighted logit derivative. $\square$

The mask depends on the request, not neural parameters. A mask that changes during differentiation would require additional analysis. Masking has a policy-gradient justification [3], but it cannot eliminate hidden server restrictions absent from the request.

Example 6.2 (Policy preference and deterministic play). *For three candidates with logits (2.55, 2.00, 0.30), Equation (14) gives probabilities approximately (0.594, 0.343, 0.063). Training samples from these probabilities. Deterministic play selects the first candidate. The number 0.594 is the probability of selecting that candidate, not a 59.4% win probability.*

6.3 How a score becomes a durable training record

For a sampled decision, the stored step contains the request identity, exact state vector, the complete action feature array, selected index and choice, collecting log probability, and collecting value estimate. An episode additionally stores format, feature schema, team identity, evidence source, collecting model revision, side, and collection mode. Storing feature arrays avoids recomputing an old observation with newly changed scouting or research data.

Live submissions are cached by room and request identity. Repeated calls normally return the cached decision. Detected server errors remove the proposal’s sample and permit another submission. Only a verified win or tie makes an episode complete; a timeout or truncation remains pending without an invented reward.

Proposition 6.3 (Scope of live idempotence). *Within one uninterrupted, serialized session, a repeated submission call for a cached request sends no second choice unless rejection handling has removed that cache entry. Repeated finalization returns the cached terminal output, and an already complete stored episode is not overwritten by ordinary finalization.*

Proof. `LiveSession.choose` checks the cached key before selecting or sending. `finish` returns an existing terminal output before processing it again. `Store.save_episode` returns when the stored identifier is already complete. These branches give the stated properties under the session and serialization assumptions. $\square$

This is not an exactly-once distributed execution theorem. The socket’s send completion is not a server acceptance acknowledgment. Rejection detection uses log errors; process crashes lose the in-memory session and no automatic mid-game reconstruction is implemented. A new request ID, a missed error, or uncoordinated external submission can defeat stronger claims.

Implementation: `ActorCritic` and `Model.predict/batch` in `ml/model.py`;
`Brain.decide/reject/finish` in `ml/brain.py`; `LiveSession.choose/finish` in `ml/live.py`;
`ml/storage.py`.

6.4 The learned residual as a change of preference

The additive prior has an exact probability interpretation. For a fixed input and candidate set, let $q(a) = \text{softmax}(q_0(a))$ and let $f_\theta(a)$ denote the neural residual logit. Then

$$\pi_\theta(a) = \frac{q(a)e^{f_\theta(a)}}{Z}, \quad Z = \sum_u q(u)e^{f_\theta(u)}. \quad (16)$$

The learned residual multiplies prior preferences before renormalization. This is sometimes called exponential tilting.

Proposition 6.4 (A variational interpretation of additive logits). *For a strictly positive prior q and finite residual scores f on a finite candidate set, $p^*(a) = q(a)e^{f(a)}/Z$ uniquely maximizes $\sum_a p(a)f(a) - D_{\text{KL}}(p||q)$ over probability distributions p .*

Proof. Since $\log p^*(a) = \log q(a) + f(a) - \log Z$,

$$\sum_a p(a)f(a) - D_{\text{KL}}(p||q) = \log Z - D_{\text{KL}}(p||p^*). \quad (17)$$

Nonnegativity of KL makes the maximum $\log Z$, attained only at $p = p^*$. $\square$

This identity interprets the forward distribution; the trainer does not explicitly optimize this variational objective or regularize toward q with a KL term. Learned residuals may overturn the tactical prior completely. Similarly, the shared scorer is equivariant to a permutation of the candidate list: permuting action rows permutes logits and probabilities. With a unique maximum, the chosen action string is invariant. Exact argmax ties retain the implementation’s list-order dependence.

7 Learning from outcomes: policy gradients, GAE, and PPO

7.1 Reward and objective

For a completed episode with T recorded decisions, let $Y \in \{-1, 0, 1\}$ denote loss, tie, or win from the learner’s perspective. The implemented rewards are $r_t = 0$ for $t < T - 1$ and $r_{T-1} = Y$. With $\gamma = 0.99$,

$$G_t = \sum_{j=t}^{T-1} \gamma^{j-t} r_j = \gamma^{T-1-t} Y. \quad (18)$$

If $\gamma = 1$, expected outcome is

$$\mathbb{E}[Y] = \mathbb{P}(\text{win}) - \mathbb{P}(\text{loss}) = 2\mathbb{P}(\text{win}) + \mathbb{P}(\text{tie}) - 1. \quad (19)$$

Maximizing it agrees with maximizing win probability only under appropriate tie conditions, such as no ties. With $\gamma < 1$, longer outcomes are discounted, and length and outcome can be correlated. Thus the training objective and the promotion rule’s raw win count are related but not identical.

The critic approximates a discounted return conditioned on its features. Its output is unrestricted and may leave $[-1, 1]$; it is not a calibrated winning probability. Hash collisions and history compression can make one feature vector correspond to different strategically relevant histories.

For a fixed policy, the ideal history-conditioned value, action value, and advantage are

$$V^\pi(h_t) = \mathbb{E}_\pi[G_t \mid h_t], \quad (20)$$

$$Q^\pi(h_t, a) = \mathbb{E}_\pi[G_t \mid h_t, a_t = a], \quad (21)$$

$$A^\pi(h_t, a) = Q^\pi(h_t, a) - V^\pi(h_t). \quad (22)$$

Thus value describes the expected outcome from a situation, action value conditions additionally on a choice, and advantage measures how much that choice departs from the situation’s policy average. The actor’s logit is not a learned Q^π estimate; its training uses advantages to adjust preferences. The critic is an approximation to value through compressed features.

7.2 The policy-gradient identity on histories

The following theorem explains the foundations, rather than asserting that every approximation in the implemented PPO optimizer is unbiased.

Theorem 7.1 (Finite-horizon likelihood-ratio gradient). *Assume a finite trajectory space and a finite horizon (pad terminated trajectories with zero-reward absorbing steps, with T the fixed padded horizon in this theorem), bounded rewards, a differentiable policy with fixed support on each history, and environment/opponent kernels independent of θ . Let $J(\theta) = \mathbb{E}_\theta[\sum_{t=0}^{T-1} \gamma^t r_t]$. Then*

$$\nabla_\theta J(\theta) = \mathbb{E}_\theta \left[\sum_{t=0}^{T-1} \gamma^t G_t \nabla_\theta \log \pi_\theta(a_t \mid h_t) \right]. \quad (23)$$

Any history-dependent action-independent baseline $B(h_t)$ can replace G_t by $G_t - B(h_t)$ without changing this expectation, treating B as fixed in the score estimator.

Proof. A trajectory probability factors into initial and transition probabilities and policy factors. Only the policy factors depend on θ , so $\nabla \log p_\theta(\tau) = \sum_t \nabla \log \pi_\theta(a_t \mid h_t)$. Differentiating the finite trajectory expectation gives the expected total return times this score. Rewards before t are measurable before a_t is sampled. Their products with the score vanish conditionally because

$$\mathbb{E}[\nabla \log \pi_\theta(a_t \mid h_t) \mid h_t] = \sum_a \nabla \pi_\theta(a \mid h_t) = \nabla 1 = 0. \quad (24)$$

The remaining rewards are $\sum_{j \geq t} \gamma^j r_j = \gamma^t G_t$. The same zero-score identity makes the baseline contribution vanish. $\square$

Interpretation. An action gains probability when it precedes an outcome better than the baseline expected for that situation. The history formulation does not require the compressed feature vector to be Markov. It does require actual sampling from the recorded distribution.

The exact discounted start-state gradient contains γ^t . The implemented PPO minibatch surrogate averages recorded decision samples equally and uses discounted advantages; it does not multiply each sample by an additional γ^t . Accordingly, its finite-batch surrogate is an optimization heuristic related to Equation (23), not an exact unbiased estimator of that particular start-state objective. Advantage standardization and an approximate critic introduce further distinctions [13–15].

7.3 Generalized advantage estimation

For collecting value estimates V_t , with $V_T = 0$, define temporal-difference residuals and advantages as

$$\delta_t = r_t + \gamma V_{t+1} - V_t, \quad (25)$$

$$\hat{A}_t = \delta_t + \gamma \lambda \hat{A}_{t+1}, \quad \hat{A}_T = 0, \quad \lambda = 0.95, \quad (26)$$

$$\hat{R}_t = \hat{A}_t + V_t. \quad (27)$$

The backward loop in `Model.train` is exactly this recurrence. The critic target $\hat{R}_t$ is a bootstrapped lambda-return; it is not generally the Monte Carlo return when $\lambda < 1$.

Proposition 7.2 (GAE expansion and the Monte Carlo endpoint). *The recurrence in Equation (27) satisfies*

$$\hat{A}_t = \sum_{j=t}^{T-1} (\gamma \lambda)^{j-t} \delta_j. \quad (28)$$

When $\lambda = 1$ and $V_T = 0$, it telescopes to $\hat{A}_t = G_t - V_t$.

Proof. Backward substitution proves the sum formula, or equivalently induction from $t = T - 1$. With $\lambda = 1$, the positive γV_{j+1} term in one residual cancels the next residual’s negative V_{j+1} after discounting. Only the discounted rewards, $-V_t$, and terminal $\gamma^{T-t} V_T$ remain; the latter is zero. $\square$

Smaller λ reduces the influence of distant residuals, often trading variance for bias when values are inaccurate. Neither this algebra nor a neural critic establishes that the encoded state is sufficient or that $\hat{A}_t$ is unbiased in this partially observed task.

Example 7.3 (Two decisions ending in a win). *Let $(V_0, V_1) = (0.20, 0.30)$ and $(r_0, r_1) = (0, 1)$. Then $\delta_1 = 0.70$ and $\delta_0 = 0.99(0.30) - 0.20 = 0.097$. Thus $\hat{A}_1 = 0.70$ and $\hat{A}_0 = 0.097 + 0.99(0.95)(0.70) = 0.75535$. The critic targets are $(0.95535, 1.00)$; the first differs from the Monte Carlo target 0.99. Normalization, when applied, changes the advantages used by the actor but not these previously computed critic targets.*

7.4 Stored probabilities and importance ratios

Let θ_{old} denote the collecting revision. For each stored candidate list and feature state, PPO computes

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid x_t, \mathcal{A}_t)}{\pi_{\theta_{\text{old}}}(a_t \mid x_t, \mathcal{A}_t)} = \exp(\log \pi_\theta(a_t \mid x_t, \mathcal{A}_t) - \text{storedLogProb}_t). \quad (29)$$

Proposition 7.4 (Conditional importance identity). *At a fixed stored input and candidate set, if old policy p is positive wherever new policy q is positive, then for any real function f on that finite set,*

$$\mathbb{E}_{a \sim p} \left[\frac{q(a)}{p(a)} f(a) \right] = \mathbb{E}_{a \sim q} [f(a)]. \quad (30)$$

Proof. Expand the expectation as $\sum_a p(a)q(a)f(a)/p(a)$ and cancel $p(a)$ on the support. The support assumption ensures no omitted term has positive q mass. $\square$

This reweights actions at fixed inputs, not the full history distribution. A single-step ratio does not correct every change in the trajectory distribution under the new policy. The denominator is also wrong if a deterministic argmax, human override, or unknown historical policy actually collected the action. This is why the harness separates sampled PPO data from demonstrations and requires a matching revision, schema, format, and completed status. Changing the candidate list after recording changes the normalization and undermines the ratio.

There is also a rejection caveat. The saved likelihood is the sampled *proposal*'s probability. If hidden restrictions cause some proposals to be rejected and only accepted proposals survive in the trajectory, then the executed-action distribution can be that proposal policy conditioned on acceptance. Unless the updated request mask fully resolves those restrictions, or the proposal/acceptance process is modeled explicitly, a stored proposal likelihood need not equal the executed-action likelihood. Excluding unfinished episodes can likewise censor trajectories if completion depends on actions. The revision and mode checks are necessary bookkeeping protections, not a proof of exact on-policy sampling under arbitrary rejections or truncations. The policy-gradient theorem applies to its stated sampling process, not automatically to every filtered live record.

7.5 The exact optimized PPO loss

With $\epsilon = 0.2$, define

$$L^{\text{clip}}(\theta) = \frac{1}{B} \sum_{t \in \mathcal{B}} \min(\rho_t \tilde{A}_t, \text{clip}(\rho_t, 0.8, 1.2) \tilde{A}_t). \quad (31)$$

If there is more than one sample and the pooled advantage standard deviation exceeds 10^{-6} , the code sets $\tilde{A}_t = (\hat{A}_t - \bar{A})/(s_A + 10^{-8})$; otherwise it leaves advantages unstandardized. The minimized batch loss is

$$\mathcal{L}(\theta) = -L^{\text{clip}}(\theta) + \frac{0.5}{B} \sum_{t \in \mathcal{B}} (V_\theta(x_t) - \hat{R}_t)^2 - \frac{0.01}{B} \sum_{t \in \mathcal{B}} H(\pi_\theta(\cdot | x_t, \mathcal{A}_t)), \quad (32)$$

where $H(\pi) = -\sum_a \pi(a) \log \pi(a)$. Entropy encourages distributed preference within the candidate set. It does not establish strategically useful exploration. The value loss fits outcomes; the actor term changes candidate probabilities.

The optimizer is Adam with learning rate $3 \cdot 10^{-4}$, minibatches of 32, four epochs by default, and global gradient norm clipped to 0.5. Epochs are restricted to 1–30. Nonfinite loss aborts before a saved checkpoint. Training produces a new revision and increments the update counter once per training call, not once per minibatch.

Proposition 7.5 (What clipping actually bounds). *For a fixed advantage A and positive ratio ρ , the clipped sample objective is*

$$\min(\rho A, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon)A) = \begin{cases} A \min(\rho, 1 + \epsilon), & A \geq 0, \\ A \max(\rho, 1 - \epsilon), & A < 0. \end{cases} \quad (33)$$

It is no larger than the sample's unclipped surrogate ρA .

Proof. For $A \geq 0$, multiplication preserves order, so the minimum selects the smaller ratio and caps only the upper side. For $A < 0$, multiplication reverses order, so the minimum selects the larger ratio and floors only the lower side. The final statement follows directly from taking a minimum with ρA . $\square$

Example 7.6 (A clipped update). *If $\pi_{\text{old}}(a) = 0.20$, $\pi_{\theta}(a) = 0.30$, and $A = 0.70$, then $\rho = 1.5$. The unclipped contribution is 1.05 but the clipped contribution is $1.2(0.70) = 0.84$. With $A = -0.70$ and $\rho = 0.5$, the clipped contribution is $0.8(-0.70) = -0.56$, not -0.35 .*

The proposition does *not* bound every probability ratio in $[0.8, 1.2]$, enforce a global KL constraint, or lower-bound actual win-rate improvement. For a positive sample the objective is already flat beyond the upper cap; shared parameters, other samples, and the critic may still move its logit. The clipped surrogate is a practical alternative to a trust-region method, not that method’s full constrained optimization or theoretical guarantee [12, 14].

7.6 Demonstrations use a different objective

For explicit demonstrations $(x_t, \mathcal{A}_t, a_t^*)$, the code minimizes

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{B} \sum_t \log \pi_{\theta}(a_t^* | x_t, \mathcal{A}_t). \quad (34)$$

This is conditional maximum likelihood. It fits labels without using the PPO ratio, entropy term, or value loss. The shared state encoder may still move, so the critic can change even though its head receives no direct BC supervision. The stored demonstration log probability is not used as a PPO denominator; the BC loss recomputes the supplied label’s probability. A reviewed human decision, an agent choice, or a heuristic teacher may supply labels, but imitating a weak teacher does not make the learner an expert.

Implementation: `Model.train` in `ml/model.py`; `Brain._train` and collection-mode checks in `ml/brain.py`; demonstration import in `ml/cli.py`.

8 The public-move scout

Public replays contain executed events, not original private requests and collecting policy probabilities. The repository therefore uses them for an auxiliary classifier rather than fabricating PPO trajectories. `ingest_public` snapshots each perspective’s public observation at the start of a turn. Later executed move events provide labels for that snapshot only if the actor was already present, its identity remains unambiguous, and the move is in the dex. Moves marked `[from]` are excluded. Chat, authentication, HTML, and private request lines are removed, and player identifiers are redacted. Incomplete public games are not ingested.

Proposition 8.1 (A turn-boundary leakage restriction). *If the supplied log is chronologically ordered, each retained scout vector is a function of the prefix through its turn boundary and the actor already visible in that prefix. Later same-turn moves and future reveals do not enter that vector.*

Proof. On a `turn` event, the importer constructs `before_turn` from the prefix and that boundary line. When a later move is encountered, it encodes the saved context; the current prefix is used only to reject ambiguous actor changes. No later prefix is substituted into the saved feature context. $\square$

This is a feature non-anticipation statement. The decision to retain or reject an example can use later identity checks, so it is not a claim that the selection of training examples is independent of future events.

Let u_t be that public feature vector with actor species and position tokens. The scout is an MLP with widths $384 \rightarrow 96 \rightarrow M$, a Tanh hidden layer, and M equal to the sorted move vocabulary size.

It minimizes cross-entropy

$$\mathcal{L}_{\text{scout}}(\psi) = -\frac{1}{N} \sum_{n=1}^N \log q_{\psi}(m_n | u_n), \quad (35)$$

using Adam at 10^{-3} , batches of 128, gradient norm cap 1, and three epochs by default. Battle digest modulo five separates held-out battles from training battles; turns from one battle cannot land in both partitions. Up to 20,000 rows are used by default. Promotion requires at least eight held-out samples and nonincreasing held-out cross-entropy. Reusing this validation partition for many candidates can still cause selection overfitting.

At inference, allowed moves come from an actually visible team sheet, or from same-format historical move labels for the species. Scout logits are normalized over this allowed subset. The three highest probabilities become hashed actor inputs. Their original probabilities are not renormalized after taking the top three, so the retained mass can be below one.

Interpretation. The scout answers: “Among moves supported by this public evidence, which executed move is likely?” It does not infer a complete hidden team or predict every submitted action, switch, target, or move prevented from executing.

8.1 Why cross-entropy is meaningful but needs qualification

Proposition 8.2 (Population cross-entropy decomposition). *For a true conditional label distribution p and a predicted distribution q positive on p ’s support,*

$$-\sum_m p(m) \log q(m) = H(p) + D_{\text{KL}}(p||q), \quad (36)$$

and this is minimized by $q = p$.

Proof. Add and subtract $-\sum_m p(m) \log p(m)$ to get the identity. For distributions, Jensen’s inequality for the concave logarithm gives $\sum_m p(m) \log(q(m)/p(m)) \leq \log \sum_{p(m)>0} q(m) \leq 0$. Hence $D_{\text{KL}}(p||q) \geq 0$, with equality at $q = p$. $\square$

This optimum concerns the label distribution of retained executed moves. Restricted inference support, selective execution, distribution shift, finite data, and limited network capacity prevent a claim of calibrated opponent intentions. A historical M-B scout can warm-start shared weights and matching move rows for Champions, but current-format validation is still required. No external expert actor checkpoint is loaded by this implementation.

Implementation: `public_lines`, `ingest_public`, and `Scout` in `ml/scout.py`; scout feature addition in `Brain.decide`.

9 Team choice: an outcome bandit over exact versions

9.1 What constitutes one team arm

A team arm is an exact ordered set list plus format, identified by

$$v = \text{SHA256}(\text{canonicalJSON}(\{\text{format}, \text{sets}\}))_{1:20}. \quad (37)$$

The identifier is twenty hexadecimal characters, i.e., an 80-bit truncated fingerprint. Canonicalization sorts object keys, not the party list. Changing a move, stat allocation, item, or ordering changes

the data being fingerprinted. This is practical versioning rather than a collision-free mathematical identity. Names are presentation labels and do not establish equivalence.

For each exact arm and evidence source, completed outcomes Y_i become scores $z_i = (Y_i + 1)/2$, giving wins 1, losses 0, and ties 1/2. Local and ladder evidence are queried separately. The estimate describes success when piloted by the policies and against the opponents present in those data, not intrinsic team quality independent of the pilot.

9.2 Beta updates and their assumptions

With a uniform prior $p_v \sim \text{Beta}(1, 1)$, the implementation uses

$$\alpha_v = 1 + \sum_i z_i, \quad \beta_v = 1 + \sum_i (1 - z_i), \quad (38)$$

and reports

$$\mu_v = \frac{\alpha_v}{\alpha_v + \beta_v}, \quad \sigma_v^2 = \frac{\alpha_v \beta_v}{(\alpha_v + \beta_v)^2 (\alpha_v + \beta_v + 1)}. \quad (39)$$

Proposition 9.1 (Bernoulli conjugacy and the tie extension). *Under conditionally independent stationary binary outcomes given p_v , w wins and l losses update a $\text{Beta}(\alpha_0, \beta_0)$ prior to $\text{Beta}(\alpha_0 + w, \beta_0 + l)$. Fractional tie updates have the same algebra under a pseudo-likelihood $\prod_i p_v^{z_i} (1 - p_v)^{1 - z_i}$, but are not an exact Bernoulli model of a three-outcome game.*

Proof. Multiplying the Beta density proportional to $p_v^{\alpha_0 - 1} (1 - p_v)^{\beta_0 - 1}$ by the Bernoulli likelihood $p_v^w (1 - p_v)^l$ gives the claimed exponents. For fractional scores, replacing integer counts by the two score sums gives identical exponent arithmetic. A Bernoulli observation takes only values zero and one, so a half-valued tie does not satisfy that sampling model. $\square$

The reported μ_v is a posterior mean win probability under the binary model. With ties, it is better interpreted as a smoothed expected score. Its uncertainty is a model-based standard deviation, not automatically a calibrated interval under evolving actor revisions, changing opponents, or correlated games. The source does not condition the team posterior on actor revision, so stationarity is an explicit limitation.

Example 9.2 (A played team and an unplayed team). *After seven wins and three losses, a team has $\text{Beta}(8, 4)$, $\mu = 2/3$, and $\sigma = \sqrt{32/1872} \approx 0.131$. An unplayed team has $\text{Beta}(1, 1)$, $\mu = 1/2$, and $\sigma = \sqrt{1/12} \approx 0.289$. The latter’s uncertainty is larger; its prior mean is not evidence that it wins half of actual battles.*

9.3 Thompson selection and constrained variants

For deterministic planning, validated candidates are ranked by μ_v , breaking ties using game count. With exploration, the planner independently samples

$$\tilde{p}_v \sim \text{Beta}(\alpha_v, \beta_v), \quad v^* = \arg \max_{v \in C} \tilde{p}_v. \quad (40)$$

This is Thompson sampling [17]. The candidate set C is bounded and validated. A separate recommendation display reports $\mu_v + \sigma_v$ as an `exploration_score`; that display score is not the Thompson selection rule.

Proposition 9.3 (Probability matching on a fixed candidate pool). *Assume independent continuous posteriors over a fixed pool C . Thompson sampling selects arm v with probability equal to the posterior probability that v has the largest sampled success parameter in C .*

Proof. The algorithm’s sample vector has exactly the joint posterior law. Its selection event is $\{\tilde{p}_v > \tilde{p}_u \ \forall u \neq v\}$. Integrating the joint posterior over this region gives the posterior probability of that same ordering. Continuity makes ties have probability zero. $\square$

This does not establish optimality over all possible Pokémon teams or a stationary-bandit regret bound for this changing pilot. The planner generally tests at most eight candidates. Variants replace one member’s complete set with an attributed same-species set from same-format research, reject duplicate items, then rely on simulator validation. Missing stat spreads are materialized as explicitly generated experimental inputs. A source’s original incomplete disclosure does not become a tournament player’s attributed complete spread.

The research prior in the battle features is also limited: it aggregates same-format imported species frequencies and overlays recent published usage. It is a bounded contextual input, not a normalized Bayesian posterior over hidden teams, and not a supervised action label. Prose is retrieved for OpenClaw to inspect rather than directly trained as battle decisions.

Implementation: `rank_teams`, `TeamPlanner.plan`, `suggest_variants`, and `generated_spreads` in `ml/teams.py`; `fingerprint` in `ml/storage.py`; `species_prior` in `ml/research.py`.

10 Background learning and the deployed model

The durable store uses SQLite in WAL mode. Jobs have persistent identities, status, attempt count, and a lease. Daily feed keys avoid redundant queue entries; practice keys allow bounded periodic collection. A worker lock prevents duplicate workers on one root, and a separate writer lock coordinates checkpoint mutation. These mechanisms improve recovery and concurrency but do not make SQLite and checkpoint replacement a single atomic distributed transaction.

The worker snapshots the incumbent checkpoint into a work directory, gathers compatible complete sampled episodes, trains a separate candidate, and evaluates incumbent and candidate separately. Unconsumed episodes must match the incumbent revision for PPO. Demonstrations are selected through their separate mode. Once a durable candidate is produced and evaluated, the consumed episodes are marked trained even if the candidate is rejected. “Trained” therefore means consumed for candidate learning, not deployed improvement.

Checkpoint files are written via a temporary file and replacement. Automatic promotion additionally requires the deployed revision still to match the original incumbent and no recent pending ladder episode of the same format. The database check uses a 30-minute recency window; this is a protective heuristic, not a proof that every possible active game is detected. Foreground `Brain.model` refuses hot reload while that format has an in-memory recorded episode. A deterministic unrecorded game does not create that same pending-episode protection. Claims of a frozen whole-game actor must account for collection mode.

The actor revision can remain frozen while scout weights or research inputs change. Consequently, “frozen actor” and “frozen decision system” mean different experimental conditions. A controlled comparison should freeze the actor, scout, knowledge, dex, teams, opponent policy, and inference mode.

10.1 Local mechanics and resource bounds

The Node bridge uses official `BattleStream`, `TeamValidator`, `Teams`, and format-specific `Dex`. Python policies receive player streams and submit complete choices through that bridge. This permits training without creating or scripting another public account. The default opponents are

a tactical heuristic and some random-policy training games; an explicit self-opponent option uses a frozen checkpoint. This is not a population league or an equilibrium-solving algorithm.

The resource policy samples CPU load, available memory, and disk. With C logical cores, current load u percent, available memory F GiB, resident worker count r , per-worker estimate m MiB, and configured limits, its worker count has the form

$$W_{\text{mem}} = \max(0, \lfloor (F - F_{\text{min}})1024/m \rfloor + r), \quad (41)$$

$$W_{\text{cpu}} = \max(0, \lfloor (u_{\text{max}} - u)C/100 \rfloor), \quad (42)$$

$$W = \max(0, \min[W_{\text{max}}, C - C_{\text{reserve}}, W_{\text{mem}}, W_{\text{cpu}}]). \quad (43)$$

Additional low-memory and low-disk checks force $W = 0$. Defaults reserve three cores and 2 GiB, cap simulator workers at four, and use a 75% host-CPU threshold. CPU versus Apple Metal (MPS) is benchmarked rather than assumed. MPS has a per-process memory fraction and deliberate pauses between minibatches. Measured load and pauses are scheduling controls, not a strict theorem about instantaneous GPU utilization or other services' latency.

Action enumeration scales with the product of per-slot option counts; preview scales as $n!/(n - k)!$. The network's action path is linear in candidate count once the fixed widths are chosen. These costs favor a small network but can still make large custom formats expensive. The implementation explicitly rejects more than two active positions.

Implementation: `ml/worker.py`, `ml/continuous.py`, `ml/resources.py`, `ml/storage.py`, `ml/simulator.py`, and `simulator.cjs`.

11 Evaluation: what would establish improvement?

11.1 The implemented promotion rule

Let n be the requested games per policy, W_C the candidate wins, W_I incumbent wins, and m the configured promotion margin. The automatic worker requires complete evaluations, no unfinished game or rejected action, and

$$W_C - W_I \geq \max(1, \lfloor nm + 0.999 \rfloor). \quad (44)$$

This is the source's exact rounding expression. It approximates a ceiling but can differ when nm is just above an integer. Defaults are $n = 20$ and $m = 0.10$, so two additional wins pass the outcome threshold. Revision and live-game checks then determine whether the file can actually be promoted.

Incumbent and candidate use matching seed, learner side, opponent, and team assignments for corresponding games. Sides alternate across the list. The current worker does not run both sides for *each* seed, and its seeds are derived from job identity rather than independently drawn from a declared population. Paired initial conditions improve comparability; they do not ensure identical later random events when different policies consume randomness differently. Auxiliary scouting and research inputs are not explicitly frozen snapshots in the task specification.

Equation (44) is an empirical acceptance heuristic. It is not a confidence interval or a hypothesis test. A candidate can win this finite suite and still be weaker on the intended deployment population. Repeated candidate selection, changing metagames, and small opponent pools intensify that concern.

11.2 A conditional statistical certificate, not implemented

The next theorem states one way to quantify stronger evidence. It is deliberately separate from the current gate.

Theorem 11.1 (A paired improvement lower bound). *Fix candidate and incumbent decision systems before evaluation. Draw n independent evaluation units from a declared target distribution. Within unit i , run both systems on matching conditions, and set $D_i = X_i^C - X_i^I \in [-1, 1]$, where X is a win indicator or bounded score in $[0, 1]$. Suppose D_i are identically distributed with mean Δ . For $\bar{D} = n^{-1} \sum_i D_i$ and $\delta \in (0, 1)$,*

$$\mathbb{P}\left(\Delta \geq \bar{D} - \sqrt{\frac{2 \log(1/\delta)}{n}}\right) \geq 1 - \delta. \quad (45)$$

Proof. Hoeffding’s inequality for independent variables in intervals of width two gives

$$\mathbb{P}(\bar{D} - \Delta \geq \eta) \leq \exp\left(-\frac{2n^2\eta^2}{\sum_i (1 - (-1))^2}\right) = \exp(-n\eta^2/2). \quad (46)$$

Choose $\eta = \sqrt{2 \log(1/\delta)/n}$ and take the complementary event [2]. $\square$

Dependence *within* a candidate/incumbent pair is allowed; independence across evaluation units is required. If both sides share one seed and are dependent, average their differences into one seed-block unit and count blocks, not individual games. The theorem concerns the declared matchup distribution; it says nothing about unrelated teams or professional tournament play.

Example 11.2 (Why a two-win margin is weak evidence). *At $n = 20$ and $\delta = 0.05$, the radius is about 0.547. An observed advantage of $2/20 = 0.10$ gives a lower bound near -0.447 , so this conservative certificate does not establish positive improvement. A radius below 0.10 requires more than $2 \log(20)/0.10^2 \approx 599.15$ independent paired units, hence at least 600. That would mean 1,200 individual games when each unit runs two policies. Other well-justified paired analyses may be more efficient; none is supplied by the current count threshold.*

For K candidates fixed independently of their evaluation data, a simultaneous bound can use δ/K for each candidate and a union bound. Adaptive tuning on the same seeds requires additional care. A final held-out evaluation should remain separate from the repeatedly used development suite. Report rejected actions and unfinished games as operational outcomes; silently discarding them can select an easier subset and bias apparent performance.

11.3 What the existing repository evidence shows

The checked-in validation documents report past experiments, not measurements performed by this paper or guarantees about the current live checkpoint. The initial ML report records 100 sampled PPO training games against one self-built mirror fixture and a scripted opponent, with a 35–65 record. Its deterministic 20-game evaluation reported 10–10 for the initial tactical prior and 16–4 for the learned policy. These small, narrow evaluations show that the pipeline can change behavior and improve that measured matchup. They do not measure broad VGC ability. The reported marginal Wilson intervals are descriptive per-policy intervals, not a paired significance test.

The later verification report documents a complete native OpenClaw Champions ladder game ending in a loss, with seven model decisions, a single terminal record, and public scout labels. It also records both rejected candidates and one small local promotion of 13/20 versus 10/20 wins. Those are integration and finite-suite observations. This paper does not infer an aggregate campaign win rate from a prompt, an update count, or an agent’s completion counter.

Implementation: Historical evidence: docs/ml-validation.md and docs/round2-validation.md.
Current gate: learn in ml/worker.py.

12 Limitations and concrete research directions

The current implementation is a compact learning player, with conditional mathematical properties and limited empirical validation. Its main unresolved issues follow directly from the model and data contracts:

1. **State sufficiency.** Expand and test protocol coverage, distinguish unknown from observed values, and assess longer histories or recurrence. An exact simulator is not enough if its observation encoder drops a mechanic.
2. **Decision quality.** Compare the tactical prior, learned residual, scout-enabled actor, and possible search or richer encoders using held-out matchups. More parameters or lower training loss is not a substitute for this.
3. **Stable data distributions.** Record scout/research versions and freeze them for evaluation. Stratify team outcomes by pilot revision or model drift if stationarity becomes untenable.
4. **Human supervision.** A conservative perspective-correct adapter is needed before external replay trajectories can become exact imitation examples. Public labels alone cannot supply on-policy PPO denominators.
5. **Statistical deployment decisions.** Use an explicit target distribution, independently held-out evaluation units, stronger opponent diversity, and uncertainty for proposed promotions.
6. **Recovery and concurrency.** Reconstruct pending sessions or clearly abandon them after crashes; strengthen submission acknowledgment and cross-process live-game protection before claiming stronger execution guarantees.

Direct VGC research such as VGC-Bench uses richer representations and training procedures [1]. That literature motivates comparisons but does not transfer its checkpoint compatibility or strength claims to this architecture. The present system has no theorem of equilibrium convergence, optimal team construction, opponent-belief recovery, or monotonic learning. Its rigor comes from making the implemented contracts and proof assumptions explicit, so each new empirical claim can be tested against a clearly defined system.

A Notation glossary

Symbol	Meaning
s_t	Complete latent simulator state; not directly supplied to the actor.
h_t	Information available to the player before a decision.
r_t, r	Reward at a decision index, and a Showdown request respectively; the request has no subscript in the action-set notation.
x_t	Encoded available information, including auxiliary features when enabled.
$\mathcal{A}_t = \mathcal{A}(r)$	Complete request-visible candidate choices.
a_t, b_t	Learner’s complete choice and opponent’s complete choice.
z_a	Action feature vector in $\mathbb{R}^{192}$.
q_0, ℓ_θ	Tactical prior and actor logit, respectively.
π_θ	Candidate selection distribution, not a win predictor.
V_t, V_θ	Collecting critic estimate and current critic function.
Y, T, G_t	Terminal outcome, recorded decision count, and discounted return.
γ, λ	Reward discount 0.99 and GAE parameter 0.95.

Symbol	Meaning
$\hat{A}_t, \tilde{A}_t$	Raw GAE estimate and advantage used by the actor.
$\hat{R}_t, \rho_t$	Lambda-return critic target and new/old likelihood ratio.
ϵ, B	PPO clipping width 0.2 and minibatch sample count.
ψ, q_ψ	Scout parameters and executed-move distribution.
v, α_v, β_v	Exact team version and its Beta shape parameters.
μ_v, σ_v	Team posterior mean and model-based standard deviation.
n, Δ, δ	Independent evaluation-unit count, true improvement, and failure probability in the proposed statistical bound.

B Source-to-equation audit map

Source	Claim or mathematical object
<code>ps_mcp_server.py</code>	Tool registration; shared client/player; asynchronous locks; lazy brain creation; team planning and model tool contracts.
<code>ps_client.py</code>	Socket login and protocol transport; own request buffering; raw choice submission.
<code>harness.py</code>	Team CRUD and packing; matchmaking; result extraction; legacy callback policy loop and fallback.
<code>battle_state.py</code>	Perspective reconstruction; preview permutations; targets; Cartesian product and joint constraint predicates.
<code>ml/features.py</code>	Schema 1; dimensions 384 and 192; signed hashing; clipping; numeric features; tactical prior in final action coordinate.
<code>ml/model.py</code>	97,826-parameter network; zero final actor initialization; masked logits; categorical sampling; GAE; PPO and BC losses and optimizer constants.
<code>ml/brain.py</code>	Knowledge/scout inputs; revision and collection-mode checks; stored feature arrays; terminal rewards; consumed-episode protection.
<code>ml/live.py</code>	Request identity cache; submission and rejection handling; verified terminal recording; full-game model loop; normal background enqueue.
<code>ml/storage.py</code>	Canonical fingerprint; SQLite tables and WAL; writer lock; episode and job status; lease-based claiming.
<code>ml/scout.py</code>	Pre-turn public labels; executed-move target; restricted inference support; cross-entropy; battle-disjoint split and scout promotion.
<code>ml/teams.py</code>	Beta estimates; Thompson selection; complete-set variants; generated stat spreads; bounded validated candidate pool.
<code>ml/research.py</code> , <code>ml/feeds.py</code>	Attributed structured evidence; species/usage inputs; source and format filtering; public replay ingestion.
<code>ml/simulator.py</code> , <code>simulator.cjs</code>	Official simulation and validation; separate player streams; local opponent policies; decision and read limits.
<code>ml/worker.py</code>	Candidate isolation; matched evaluations; empirical win margin; current-revision and recent live-game checks; atomic promotion.
<code>ml/continuous.py</code> , <code>ml/resources.py</code> <code>policy.py</code>	Durable scheduling; terminal observer; default run limits; worker headroom; CPU/MPS benchmarking. Earlier scripted type-aware policy; not the neural architecture described by the PPO equations.

C Reproducibility and document build

Build this report from `docs/paper/` using `tectonic main.tex`, or a conventional `latexmk -pdf main.tex`. The adjacent bibliography is a separate BibTeX file. No battle credentials, model weights, raw replay archive, or private machine paths are required to compile the document. Supporting source references are recorded in `docs/paper-references.md`.

To reproduce a model experiment, record at least:

1. Repository and simulator revisions, exact format, dependency versions, feature schema, device, and random seeds.
2. Exact teams and ordering, generated-spread flags and source attribution, learner side, and opponent policy/version.
3. Actor, scout, research and dex snapshots; collection mode; exploration setting; terminal, unfinished and rejected-action counts.
4. Collected revision and stored feature/action arrays; training hyperparameters; candidate revision; consumed data identifiers; promotion report.
5. Independent evaluation units, matchup distribution, side balancing, uncertainty method, and separation of local versus ladder evidence.

Use an isolated data root for experiments. The ordinary root is persistent learning state, and an active campaign can change it during a purported frozen comparison. Run the existing offline tests for action generation, masking, collection modes, rejection handling, terminal recording, revision reload, pre-turn scout features, and source isolation. Passing tests supports these implemented contracts; it is not a measurement of competitive playing strength.

References

- [1] Cameron L. Angliss, Jiaxun Cui, Jiaheng Hu, Arrasy Rahman, and Peter Stone. VGC-Bench: Towards mastering diverse team strategies in competitive Pokémon. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS, 2026. URL <https://www.cs.utexas.edu/~pstone/Papers/bib2html-links/angliss2026vgc.pdf>.
- [2] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. doi: 10.1080/01621459.1963.10500830. URL <https://doi.org/10.1080/01621459.1963.10500830>.
- [3] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. *The International FLAIRS Conference Proceedings*, 35, 2022. doi: 10.32473/flairs.v35i.130584. URL <https://arxiv.org/abs/2006.14171>.
- [4] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X. URL <https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf>.
- [5] Model Context Protocol contributors. Model context protocol specification: Tools, 2025. URL <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>. Specification version 2025-06-18; accessed 2026-10-07.
- [6] Model Context Protocol contributors. Model context protocol specification: Transports, 2025. URL <https://modelcontextprotocol.io/specification/2025-06-18/basic/transports>. Specification version 2025-06-18; accessed 2026-10-07.

- [7] OpenClaw contributors. Manage saved mcp servers, n.d. URL <https://docs.openclaw.ai/cli/mcp/registry>. Accessed 2026-10-07.
- [8] Pokémon Showdown contributors. Battlestream and player stream implementation, n.d.. URL <https://github.com/smogon/pokemon-showdown/blob/c046106cbe075931b1ff8d8b800ff5be47a85f96/sim/battle-stream.ts>. Pinned official simulator source; inspected through repository dependency.
- [9] Pokémon Showdown contributors. Pokémon showdown client-server protocol, n.d.. URL <https://github.com/smogon/pokemon-showdown/blob/c046106cbe075931b1ff8d8b800ff5be47a85f96/PROTOCOL.md>. Pinned simulator source; inspected through repository dependency.
- [10] Pokémon Showdown contributors. Pokémon showdown: Pinned simulator and format source, n.d.. URL <https://github.com/smogon/pokemon-showdown/tree/c046106cbe075931b1ff8d8b800ff5be47a85f96>. Commit c046106cbe075931b1ff8d8b800ff5be47a85f96; project dependency pin.
- [11] Pokémon Showdown contributors. Pokémon showdown simulator protocol, n.d.. URL <https://github.com/smogon/pokemon-showdown/blob/master/sim/SIM-PROTOCOL.md>. Accessed 2026-10-07.
- [12] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897, 2015. URL <https://proceedings.mlr.press/v37/schulman15.html>.
- [13] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016. URL <https://arxiv.org/abs/1506.02438>.
- [14] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. doi: 10.48550/arXiv.1707.06347. URL <https://arxiv.org/abs/1707.06347>.
- [15] Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html.
- [16] The Pokémon Company. Regulation set m-c (updated on september 9), 2026. URL <https://champions-news.pokemon-home.com/en/page/816.html>. Accessed 2026-10-07.
- [17] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933. doi: 10.1093/biomet/25.3-4.285. URL <https://academic.oup.com/biomet/article-abstract/25/3-4/285/200862>.
- [18] Kilian Weinberger, Anirban Dasgupta, Josh Attenberg, John Langford, and Alex Smola. Feature hashing for large scale multitask learning. *arXiv preprint arXiv:0902.2206*, 2009. URL <https://arxiv.org/abs/0902.2206>. Version 5 revised 2010.